

MASTER 2 MIAGE — Université Paris-Saclay

Unité d'Enseignement — Projet DATA

CAHIER DES CHARGES
Simulateur de Portefeuille Passif
avec Régression Linéaire Prédictive

Encadrant	Nicolas LEGEAY
Groupes	Binômes ou trinômes (2 ou 3 étudiants)
Durée	6 à 8 semaines
Livrables	Application web déployée + Rapport écrit (15–20 pages) + Soutenance
Année universitaire	2025–2026

Table des matières

Table des matières	2
1. Présentation du projet	4
1.1 Contexte	4
1.2 L'investissement passif — ce que vous devez savoir	4
1.3 Objectifs pédagogiques	4
2. Ce que vous devez construire	6
2.1 Vue d'ensemble	6
2.2 Module A — Explorateur d'ETF	6
Fonctionnalités attendues	6
Données à utiliser	6
2.3 Module B — Simulateur DCA (backtesting)	6
Paramètres configurables par l'utilisateur	7
Résultats affichés	7
Algorithme à implémenter	7
2.4 Module C — Régression linéaire et analyse de tendance	7
Fonctionnalités à implémenter	8
L'interprétation critique — obligatoire	8
3. Architecture technique	9
3.1 Stack technologique recommandée	9
3.2 Modèle de données	9
3.3 Endpoints API à exposer	10
4. Planning et organisation	11
4.1 Répartition des rôles	11
4.2 Planning recommandé — 6 semaines	11
4.3 Points de contrôle intermédiaires	12
5. Spécifications analytiques détaillées	13
5.1 Calcul du CAGR (rendement annualisé)	13
5.2 Impact des frais TER — formule	13
5.3 Régression linéaire — spécifications complètes	13
Variables du modèle	13
Calculs à produire	13
Affichages attendus	13
Exemple de résultat attendu pour CW8 (10 ans de données)	14
5.4 Exigences de qualité du code	14
6. Livrables attendus	15
6.1 Application web déployée	15
6.2 Rapport écrit (15 à 20 pages, rendu en PDF 48h avant la soutenance)	15
6.3 Soutenance (20 minutes + 10 minutes de questions)	16

6.4 Dépôt GitHub.....	16
7. Grille d'évaluation.....	17
8. Ressources utiles.....	18
8.1 Pour démarrer rapidement.....	18
8.2 Déploiement gratuit	18
8.3 Pour comprendre l'investissement passif	18
8.4 Pour aller plus loin sur la régression	18

1. Présentation du projet

1.1 Contexte

Dans le cadre de l'UE Projet DATA, vous allez concevoir et déployer une application web permettant de simuler et d'analyser des stratégies d'investissement passif en bourse. Ce projet est volontairement ancré dans un cas d'usage concret et professionnel : un investisseur particulier qui souhaite comprendre comment placer son épargne de façon rationnelle et peu coûteuse.

Vous n'avez pas besoin d'être expert en finance. Les notions nécessaires sont introduites dans ce document. En revanche, on attend de vous une rigueur technique de niveau M2 MIAGE : architecture propre, code documenté, données réelles, déploiement fonctionnel.

1.2 L'investissement passif — ce que vous devez savoir

Avant de coder, prenez le temps de comprendre ce que votre application va modéliser. Voici les trois notions clés du projet :

ETF (Exchange-Traded Fund)

Un ETF est un fonds coté en bourse qui réplique automatiquement un indice de marché (ex. le S&P 500 regroupe les 500 plus grandes entreprises américaines). Quand le S&P 500 monte de 10 %, votre ETF monte de ~10 %. Vous n'avez pas à choisir vous-même les actions : vous achetez "le marché entier". C'est la base de l'investissement passif.

DCA — Dollar Cost Averaging (investissement programmé)

Plutôt que d'investir toute son épargne d'un coup (risque de "tout mettre au mauvais moment"), le DCA consiste à investir une somme fixe chaque mois, quelle que soit la situation du marché. Si le cours baisse, vous achetez plus de parts. Si le cours monte, vous en achetez moins. Sur le long terme, cela lisse le prix moyen d'achat. C'est la stratégie recommandée aux investisseurs individuels.

TER — Total Expense Ratio (frais annuels)

Le TER est le coût annuel prélevé automatiquement par le gestionnaire du fonds, exprimé en % du capital investi. Un ETF monde coûte ~0,20 %/an. Un fonds géré activement coûte ~1,5 %/an. Sur 30 ans, cette différence de 1,3 %/an peut représenter des dizaines de milliers d'euros de différence sur un portefeuille moyen. Votre simulateur devra le démontrer.

1.3 Objectifs pédagogiques

Ce projet vous permet de mobiliser simultanément vos compétences techniques et de développer une culture financière concrète. À l'issue du projet, vous devrez être capables de :

- Construire un pipeline ETL complet (collecte → transformation → stockage) sur des données financières réelles
- Exposer des traitements métier via une API REST documentée

- Implémenter et interpréter une régression linéaire simple appliquée à des séries temporelles
- Développer une interface React lisible, avec des visualisations graphiques pertinentes
- Déployer votre application en production sur une URL publique
- Expliquer, à l'oral comme à l'écrit, pourquoi l'investissement passif est une stratégie rationnelle — et quelles sont les limites de vos propres modèles

2. Ce que vous devez construire

2.1 Vue d'ensemble

Votre application s'articule autour de trois modules. Le troisième (régression) est le plus exigeant analytiquement : c'est celui sur lequel vous serez le plus questionnés en soutenance.

Module A — Explorateur d'ETF
Régression linéaire

Module B — Simulateur DCA

Module C —

Périmètre calibré pour un binôme / trinôme

Ces trois modules sont le périmètre obligatoire. Ils sont conçus pour être réalisables en 6 à 8 semaines à 2 ou 3 personnes, à condition de bien découper le travail dès le départ. Ne cherchez pas à ajouter des fonctionnalités supplémentaires avant d'avoir les trois modules fonctionnels avec de vraies données.

2.2 Module A — Explorateur d'ETF

Ce module permet à l'utilisateur de rechercher un ETF et d'en comprendre les caractéristiques avant d'investir. C'est le point d'entrée de l'application.

Fonctionnalités attendues

- Recherche d'un ETF par son nom ou son ticker boursier (ex. "CW8", "World")
- Affichage de la fiche descriptive : nom complet, indice répliqué, gestionnaire, TER
- Mention explicite de l'éligibilité PEA (enveloppe fiscale française avantageuse)
- Graphique interactif de l'évolution du cours sur 1 an, 3 ans et 10 ans
- Comparaison côte à côte de deux ETF : performance et frais

Données à utiliser

- Cours historiques : bibliothèque Python yfinance (gratuit, sans clé API)
- Métadonnées (TER, indice, éligibilité PEA) : fichier CSV fourni ou scraping JustETF

ETF de référence pour ce projet

Vous travaillerez avec au minimum ces 4 ETF : CW8 (Amundi MSCI World, TER 0,38 %, éligible PEA), PS20 (Amundi S&P 500, TER 0,15 %, éligible PEA), ESE (iShares MSCI Europe, TER 0,12 %), OBLI (Lyxor Obligations d'État Euro, TER 0,17 %). Commencez toujours par CW8 pour valider votre pipeline.

2.3 Module B — Simulateur DCA (backtesting)

C'est le cœur de l'application. L'utilisateur configure sa stratégie d'investissement et voit comment son portefeuille aurait évolué sur des données historiques réelles. On appelle cela du backtesting : tester une stratégie sur le passé.

Paramètres configurables par l'utilisateur

- Sélection d'un ETF parmi ceux disponibles
- Capital de départ (ex. 1 000 €)
- Versement mensuel DCA (ex. 200 €/mois)
- Période de simulation (ex. janvier 2015 → décembre 2024)
- TER de l'ETF (pré-rempli, modifiable)

Résultats affichés

- Courbe de l'évolution du portefeuille sur la période (valeur totale en €)
- Tableau comparatif : valeur finale avec frais réels vs sans frais — pour illustrer l'impact du TER
- Métriques clés : capital total versé, valeur finale, gain net, rendement annualisé (CAGR)
- Courbe secondaire : évolution du même capital investi sur un livret A (taux fixe) → comparaison avec l'épargne sans risque

Algorithme à implémenter

Voici la logique mois par mois que vous devez coder :

1. Récupérer le prix de clôture du premier jour de trading du mois
2. Calculer le nombre de parts achetées = versement mensuel ÷ prix du jour
3. Ajouter ces parts au total cumulé
4. Déduire les frais TER mensualisés : valeur du portefeuille × (1 – TER ÷ 12)
5. Calculer la valeur totale = total des parts × prix du jour
6. Stocker la valeur dans la base de données

Pourquoi le backtesting n'est pas une garantie

Votre rapport devra mentionner explicitement ce point : le fait qu'une stratégie ait bien fonctionné dans le passé ne garantit pas ses performances futures. Le backtesting est un outil d'illustration, pas de prédiction. C'est justement pourquoi l'investissement passif régulier (DCA) est préférable au market timing.

2.4 Module C — Régression linéaire et analyse de tendance

Ce module constitue la valeur ajoutée analytique de votre projet. Vous y appliquez une régression linéaire simple (OLS) sur les données historiques d'un ETF afin d'analyser sa tendance de long terme et d'en tirer des enseignements sur les limites de la prédiction financière.

Avant de coder, assurez-vous de comprendre ce que vous modélisez :

La régression linéaire simple — rappel

Vous cherchez à modéliser la relation entre deux variables : le temps (X , en nombre de jours de trading depuis le début) et le prix de clôture d'un ETF (Y). La droite de régression $Y = \beta_0 + \beta_1 \cdot X$ représente la tendance moyenne. β_1 est la pente : s'il est positif, l'ETF a une tendance haussière. R^2 mesure à quel point la droite explique les variations de prix (entre 0 et 1).

Fonctionnalités à implémenter

- Sélection d'un ETF et d'une fenêtre d'analyse (entre 3 et 15 ans de données)
- Calcul des coefficients β_0 et β_1 par OLS via scikit-learn ou NumPy
- Graphique du cours historique avec superposition de la droite de régression
- Intervalles de confiance à 95 % autour de la droite
- Tableau de synthèse des métriques : R^2 , pente β_1 en €/jour et en %/an, p-value
- Graphique des résidus (cours réel – cours prédit) en fonction du temps
- Projection de la droite sur 12 mois futurs (à titre illustratif uniquement, avec mention claire de cette limite)

L'interprétation critique — obligatoire

Votre application doit afficher une section d'analyse sous les graphiques. Voici les points que cette section doit couvrir, que vous devrez aussi défendre à l'oral :

- Un R^2 élevé sur longue période confirme la tendance haussière des marchés → argument en faveur de l'investissement passif
- Les résidus ne sont pas aléatoires : ils présentent des cycles, des crises, des bulles → le marché n'est pas prévisible à court terme
- L'intervalle de confiance s'élargit rapidement dans le futur → la projection est de moins en moins fiable
- Conclusion attendue : "La régression confirme une tendance, pas un timing. C'est pourquoi le DCA est plus rationnel que d'essayer de trouver le bon moment pour investir."

Attention à la régression fallacieuse (spurious regression)

Sur des séries temporelles financières, un R^2 proche de 1 ne signifie pas que votre modèle est bon. Deux séries qui montent toutes les deux dans le temps auront toujours un R^2 élevé, même si elles n'ont aucun lien causal. Mentionnez ce risque dans votre rapport : c'est ce qui distingue une analyse sérieuse d'une analyse naïve.

3. Architecture technique

3.1 Stack technologique recommandée

Vous avez la liberté de vos choix techniques, à condition que les justifications apparaissent dans votre rapport. Les technologies ci-dessous sont recommandées car elles correspondent à votre formation et permettent un déploiement gratuit.

Couche	Technologie recommandée	Rôle
Collecte de données	Python + yfinance	Récupération des cours historiques depuis Yahoo Finance
Traitement	Pandas, NumPy	Nettoyage, calculs financiers, préparation des séries temporelles
Modèle statistique	scikit-learn (LinearRegression)	Régression OLS, calcul R^2 , résidus, intervalles de confiance
Base de données	PostgreSQL	Stockage des cours, des ETF et des résultats de simulation
Backend / API	FastAPI (Python)	Exposition des traitements via REST, documentation Swagger auto-générée
Frontend	React 18 + Vite	Interface utilisateur, navigation entre les modules
Graphiques	Recharts ou Plotly.js	Visualisations interactives (zoom, hover, export)
Déploiement backend	Railway ou Render	Hébergement gratuit FastAPI + PostgreSQL
Déploiement frontend	Vercel	Déploiement automatique depuis GitHub, CDN mondial
Versioning	Git + GitHub	Un dépôt par groupe, une branche par module

3.2 Modèle de données

Voici le modèle relationnel minimal attendu. Vous pouvez l'enrichir, mais toutes ces tables doivent être présentes :

Table	Colonnes principales
etf	id, ticker, nom, indice_replique, gestionnaire, ter, eligible_pea
cours_historique	id, etf_id (FK), date, prix_cloture_ajuste, volume
simulation	id, etf_id (FK), capital_initial, versement_mensuel, date_debut, date_fin, created_at
resultat_simulation	id, simulation_id (FK), date, valeur_portefeuille, parts_cumulees, frais_cumules
resultat_regression	id, etf_id (FK), fenetre_annees, r2, pente_jour, p_value, created_at

3.3 Endpoints API à exposer

Votre API REST doit exposer au minimum les routes suivantes. Elles doivent toutes être documentées dans Swagger (accessible à /docs sur votre backend FastAPI) :

Méthode + Route	Description
GET /etf/	Liste des ETF disponibles dans l'application
GET /etf/{ticker}	Fiche complète d'un ETF (métadonnées + derniers cours)
GET /etf/{ticker}/historique	Série temporelle des cours (paramètres : date_debut, date_fin)
POST /simulation/	Lance un backtesting DCA avec les paramètres du body JSON
GET /simulation/{id}	Résultats complets d'une simulation existante
POST /regression/	Lance une régression linéaire sur un ETF (fenêtre paramétrable)
GET /regression/{etf_id}	Derniers résultats de régression disponibles pour un ETF

4. Planning et organisation

4.1 Répartition des rôles

Avec 2 ou 3 personnes, il n'y a pas de rôles figés. En revanche, chaque membre doit toucher au backend ET au frontend : ce projet n'est pas découparable en "celui qui code" et "celui qui fait le rapport". Voici une répartition suggérée :

Rôle partagé	Responsabilité principale
Data & Backend (tous)	Pipeline ETL, modèle de données, API FastAPI — à faire ensemble ou en binôme
Analyse statistique	Régression linéaire, interprétation des métriques, graphiques résidus
Frontend & intégration	Interface React, intégration des graphiques, UX, déploiement Vercel

Conseil d'organisation
Commencez la semaine 1 par un seul ETF (CW8) et faites fonctionner l'ensemble du pipeline de bout en bout : collecte → stockage → API → affichage sur une page React. Une fois ce "squelette" validé, généraliser à d'autres ETF et ajouter les modules sera beaucoup plus rapide.

4.2 Planning recommandé — 6 semaines

Sprint	Semaine	Objectif	Livrables attendus
S1	Sem. 1	Socle technique & données	Environnement configuré (Python, PostgreSQL, React). Script yfinance fonctionnel. Données de CW8 stockées en BDD. Schéma de données validé avec l'encadrant.
S2	Sem. 2	API Backend — Module A	Endpoints /etf/ et /etf/{ticker}/historique opérationnels. Swagger accessible. Fiche ETF affichée en React avec graphique du cours.
S3	Sem. 3	Simulateur DCA — Module B	Algorithme DCA codé et testé sur CW8. Endpoint /simulation/ fonctionnel. Courbe de portefeuille affichée. Comparaison avec/sans frais TER visible.
S4	Sem. 4	Régression linéaire — Module C	Modèle OLS implémenté. Métriques calculées (R^2 , pente, p-value). Graphique cours + droite + intervalles. Section d'interprétation critique rédigée.
S5	Sem. 5	Intégration & polish	Navigation fluide entre les 3 modules. Application cohérente avec 4 ETF minimum. Tests manuels complets. Correction des bugs.
S6	Sem. 6	Déploiement & rapport	Application déployée sur URL publique. Rapport écrit finalisé (PDF). Slides de soutenance prêtes. README GitHub complet.

Version 8 semaines

Si vous avez 8 semaines, utilisez les semaines 7 et 8 pour : affiner l'interprétation de la régression (test de Durbin-Watson, analyse des résidus plus poussée), ajouter la comparaison ETF passif vs fonds actif (données publiques), et améliorer l'expérience utilisateur.

4.3 Points de contrôle intermédiaires

Moment	Ce que vous montrez à l'encadrant
Fin semaine 1	Le schéma de données (diagramme) et la collecte yfinance fonctionnelle sur CW8
Fin semaine 3	La simulation DCA complète sur CW8 avec courbe affichée en React
Fin semaine 5	L'application complète et navigable avec données réelles sur 4 ETF
Soutenance	Application déployée, rapport rendu 48h avant, démonstration live obligatoire

5. Spécifications analytiques détaillées

5.1 Calcul du CAGR (rendement annualisé)

Le CAGR (Compound Annual Growth Rate) mesure le taux de croissance annuel moyen d'un investissement sur plusieurs années. C'est la métrique principale pour comparer des stratégies sur des périodes différentes. Voici la formule :

$$\text{CAGR} = (\text{Valeur finale} \div \text{Capital total versé})^{(1 \div \text{nombre d'années})} - 1$$

Exemple : vous avez versé 15 000 € sur 5 ans (DCA) et votre portefeuille vaut 21 000 €. $\text{CAGR} = (21\,000 / 15\,000)^{(1/5)} - 1 \approx 6,96\%/\text{an}$. Affichez ce résultat avec deux décimales et expliquez-le à l'utilisateur en langage clair.

5.2 Impact des frais TER — formule

Pour calculer la valeur du portefeuille avec frais, vous déduisez le TER mensuellement. La formule à appliquer à chaque mois de simulation est :

$$\text{Valeur après frais (mois } n) = \text{Valeur (mois } n) \times (1 - \text{TER} \div 12)$$

Vous devez afficher deux courbes sur le même graphique : l'une avec le TER réel de l'ETF, l'autre avec $\text{TER} = 0$. La différence finale illustrera de façon concrète pourquoi les frais sont si importants sur le long terme.

5.3 Régression linéaire — spécifications complètes

Variables du modèle

- X (variable explicative) : numéro du jour de trading depuis le début de la fenêtre (0, 1, 2, ... N)
- Y (variable à expliquer) : prix de clôture ajusté de l'ETF ce jour-là

Calculs à produire

- Coefficients β_0 et β_1 (via `sklearn.linear_model.LinearRegression` ou `numpy.polyfit`)
- R^2 — coefficient de détermination (via `score()` de `scikit-learn` ou calcul manuel)
- P-value sur β_1 — test que la pente est significativement différente de zéro (via `scipy.stats.linregress` ou `statsmodels`)
- Série des résidus : $e_i = Y_i - (\beta_0 + \beta_1 \cdot X_i)$
- Intervalle de confiance à 95 % sur la droite (via `statsmodels.OLS` ou calcul manuel de l'erreur standard)

Affichages attendus

- Graphique 1 : cours historique (bleu) + droite de régression (orange) + bande de confiance 95 % (gris clair)
- Graphique 2 : résidus en fonction du temps — la série doit osciller autour de zéro
- Tableau de synthèse : R^2 , pente en €/jour, pente annualisée en %/an, p-value, interprétation en une phrase

Exemple de résultat attendu pour CW8 (10 ans de données)

Indicateur	Valeur typique	Ce que ça signifie
R^2	~0,88 à 0,92	88–92 % de la variance du prix expliquée par le temps → tendance forte
Pente β_1	~+0,06 €/jour	Le prix moyen augmente d'environ 6 centimes par jour de trading
Pente annualisée	~+9 à +12 %/an	Cohérent avec la performance historique du MSCI World
P-value	< 0,001	La tendance est statistiquement très significative
Résidus	Non aléatoires	Signe d'autocorrélation → marché efficient, non prédictible à court terme

5.4 Exigences de qualité du code

- Toutes les clés de configuration (URL de BDD, tokens) dans un fichier .env — jamais dans le code versionné
- Un fichier .gitignore complet dès le premier commit (inclure .env, __pycache__, node_modules)
- Chaque endpoint FastAPI est documenté (description dans le décorateur ou docstring)
- Au moins 3 tests unitaires sur le moteur de simulation DCA (pytest) — testez des cas limites : capital initial = 0, TER = 0, période d'un seul mois
- README clair avec : comment lancer le projet en local, comment configurer la base de données, lien vers l'application déployée

6. Livrables attendus

6.1 Application web déployée

L'application doit être accessible via une URL publique stable au moment de votre soutenance. Voici ce qui est évalué :

- Les 3 modules sont accessibles et fonctionnent avec de vraies données (pas des données fictives hardcodées)
- La navigation entre les modules est fluide et intuitive
- Les graphiques sont lisibles et interactifs (zoom, survol des valeurs)
- L'application ne plante pas en conditions normales d'utilisation
- Le temps de réponse est raisonnable : une simulation doit s'afficher en moins de 5 secondes

6.2 Rapport écrit (15 à 20 pages, rendu en PDF 48h avant la soutenance)

Votre rapport doit suivre ce plan. Chaque section doit être rédigée en vos propres mots — le rapport n'est pas une liste de captures d'écran.

Section	Contenu attendu	Volume indicatif
1. Introduction	Contexte, problématique, scope du projet, organisation du rapport	1 page
2. Notions financières mobilisées	ETF, DCA, TER, backtesting — expliquez ces concepts comme si vous les présentiez à quelqu'un qui ne les connaît pas	2 pages
3. Architecture & choix techniques	Diagramme d'architecture, justification du stack, modèle de données commenté	2–3 pages
4. Pipeline de données	Sources utilisées, processus ETL, qualité des données, fréquence de mise à jour	2 pages
5. Simulateur DCA	Présentation de l'algorithme, cas de test, résultats sur 2 ETF minimum, analyse de l'impact des frais	2–3 pages
6. Régression linéaire	Méthodologie, résultats sur 2 ETF (tableaux + graphiques), interprétation critique développée	3–4 pages
7. Conclusion	Limites du projet, ce que vous feriez différemment, bilan personnel sur l'investissement passif	1 page
Annexes	Lien GitHub, lien application déployée, documentation Swagger (captures)	hors limite

Ce qu'on attend vraiment de votre rapport

La section sur la régression est la plus discriminante. Ne vous contentez pas d'afficher $R^2 = 0,91$ et d'écrire "le modèle est bon". Expliquez pourquoi ce R^2 élevé est en réalité attendu sur une série temporelle croissante, pourquoi cela ne signifie pas que vous pouvez prédire les cours, et ce que les résidus vous apprennent sur l'efficience des marchés. C'est ce niveau de recul qui sera valorisé.

6.3 Soutenance (20 minutes + 10 minutes de questions)

- Présentation structurée : contexte (2 min), architecture (3 min), démonstration live (10 min), résultats et analyse (5 min)
- La démonstration live est obligatoire — préparez un scénario de démonstration précis à l'avance
- Chaque membre du groupe présente une partie — la répartition doit être équilibrée
- Les questions porteront notamment sur : la régression et son interprétation, les choix d'architecture, les limites du projet

6.4 Dépôt GitHub

- Un dépôt par groupe, partagé avec l'encadrant dès la semaine 1
- Historique de commits progressif — un seul commit géant la veille de la soutenance est rédhibitoire
- README complet : prérequis, installation, lancement en local, lien vers le déploiement
- Fichier .env.example (avec les variables nécessaires mais sans les valeurs sensibles)

7. Grille d'évaluation

Critère	Sous-critère	/ pts	Attendu
Application déployée	Fonctionnalité — 3 modules opérationnels avec données réelles	6	Tous les modules fonctionnent, les données sont vraies, l'app est en ligne
Application déployée	Qualité technique — code, architecture, tests	3	Code propre, .env, tests unitaires présents, README complet
Application déployée	Interface — lisibilité et graphiques	2	Navigation claire, graphiques lisibles et interactifs, pas de bugs visuels
Rapport écrit	Clarté et structure	2	Plan respecté, rédaction soignée, figures légendées
Rapport écrit	Analyse de la régression	4	Métriques correctes, interprétation critique développée, limites identifiées
Soutenance	Présentation et démonstration live	2	Clarté, démonstration sans accroc, réponses aux questions
Soutenance	Maîtrise individuelle	1	Chaque membre maîtrise sa partie et peut répondre aux questions

Bonus — jusqu'à +2 points sur 20

Test de Durbin-Watson implémenté avec interprétation rédigée (+0,5 pt). Comparaison ETF passif vs fonds actif réel avec données publiques (+1,5 pt). Système de sauvegarde des simulations utilisateur (JWT + BDD) (+0,5 pt). Visualisation avancée : animation de la progression du portefeuille, export PDF du rapport de simulation (+0,5 pt).

8. Ressources utiles

8.1 Pour démarrer rapidement

- yfinance — récupérer les cours en 3 lignes de Python : `pip install yfinance` puis `import yfinance as yf; yf.download("CW8.PA", start="2015-01-01")`
- FastAPI — tutoriel officiel : <https://fastapi.tiangolo.com/tutorial/> (lisez au moins les sections "First Steps" et "SQL Databases")
- scikit-learn LinearRegression : https://scikit-learn.org/stable/modules/linear_model.html — rubrique "Ordinary Least Squares"
- Recharts — bibliothèque de graphiques React : <https://recharts.org/en-US/examples> — les exemples LineChart et ComposedChart sont ceux dont vous aurez besoin

8.2 Déploiement gratuit

- Vercel (frontend React) : <https://vercel.com> — connectez votre dépôt GitHub, le déploiement est automatique à chaque push
- Railway (backend FastAPI + PostgreSQL) : <https://railway.app> — 5 \$ de crédit offert par mois, largement suffisant pour ce projet
- Render (alternative à Railway) : <https://render.com> — tier gratuit disponible, un peu plus lent au démarrage

8.3 Pour comprendre l'investissement passif

- JustETF — screener d'ETF Europe (filtres PEA, TER, indice) : <https://www.justetf.com/fr/>
- AMF — guide de l'investisseur particulier : <https://www.amf-france.org/> (section "Épargner et investir")
- SPIVA Europe Scorecard (S&P Global) — données annuelles sur la sous-performance des fonds actifs vs indices : cherchez "SPIVA Europe scorecard" sur [spglobal.com](https://www.spglobal.com)

8.4 Pour aller plus loin sur la régression

- statsmodels — librairie Python pour l'analyse statistique avancée (OLS avec p-values détaillées) : <https://www.statsmodels.org/>
- "Spurious regressions in econometrics" — Granger & Newbold (1974) : article fondateur sur la régression fallacieuse dans les séries temporelles financières

Bonne chance à tous — n'hésitez pas à me solliciter dès que vous êtes bloqués, et surtout : commencez par faire fonctionner quelque chose de simple avant de vouloir tout construire en même temps.
